

Table of Contents

1. [Docker — Basics](#section-1-docker--basics) (12 questions)
2. [Docker — Intermediate](#section-2-docker--intermediate) (8 questions)
3. [Docker — Advanced](#section-3-docker--advanced) (6 questions)
4. [Kubernetes — Basics](#section-4-kubernetes--basics) (12 questions)
5. [Kubernetes — Intermediate](#section-5-kubernetes--intermediate) (9 questions)
6. [Kubernetes — Advanced](#section-6-kubernetes--advanced) (10 questions)
7. [Scenario-Based / Troubleshooting](#section-7-scenario-based--troubleshooting) (11 questions)
8. [Project-Specific — Banking App on K8s](#section-8-project-specific--banking-app-on-k8s) (7 questions)

Total: 75 questions

Section 1: Docker — Basics

1. What is Docker and how does it differ from a virtual machine?

Answer:

Docker is a platform for packaging and running applications in lightweight, isolated containers. Unlike VMs, containers share the host OS kernel rather than running a full guest OS.

	VM	Container
OS	Full guest OS + hypervisor	Shares host kernel
Size	GBs	MBs
Startup	Minutes	Seconds
Isolation	Strong (separate kernel)	Process-level (namespaces + cgroups)
Use case	Legacy apps, full OS testing	Microservices, CI/CD, cloud-native

2. What is the difference between a Docker Image and a Docker Container?

Answer:

- Image: A read-only, layered template built from a Dockerfile. Think of it as a **class** in OOP — static, reusable, stored in a registry.
- Container: A running instance of an image. Think of it as an **object** — dynamic, has a writable top layer, lives in memory.

Multiple containers can run from the same image simultaneously. An image becomes a container when you run `docker run <image>`.

3. What is a Dockerfile? Walk me through the common instructions.

Answer:

A Dockerfile is a text file with step-by-step instructions to build a Docker image.

Instruction	Purpose	Example
FROM	Base image to build upon	FROM node:20-alpine
WORKDIR	Set working directory	WORKDIR /app
COPY	Copy files from host to image	COPY package*.json ./
ADD	Like COPY + supports URLs + auto-extracts tar	ADD app.tar.gz /app
RUN	Execute command during build (creates a layer)	RUN npm install
ENV - running container	Set environment variables	ENV PORT=3000
EXPOSE	Document which port app uses	EXPOSE 8080
CMD	Default command (overridable at runtime)	CMD ["node", "server.js"]
ENTRYPOINT	Fixed main process	ENTRYPOINT ["java", "-jar"]
ARG - building image	Build-time variable	ARG VERSION=1.0
USER	Switch to non-root user	USER appuser

Why alpine? Minimal packages, tiny image,

How to install a package in alpine? `Apk add --no-cache curl`

Alternatives: debian-slim/ubuntu, distroless

4. What is the difference between CMD and ENTRYPOINT?

Answer:

- ENTRYPOINT defines the executable that always runs. Cannot be overridden at runtime (only with `--entrypoint` flag).
- CMD provides default arguments that can be fully overridden by passing arguments to `docker run`.

```
ENTRYPOINT ["java", "-jar"]
CMD ["app.jar"]
# Runs: java -jar app.jar
# Override: docker run myimage other.jar → java -jar other.jar
```

5. What is Docker Hub? How do you push and pull images? What are the alternatives?

Answer:

Docker Hub is the default public container registry — like GitHub but for Docker images.

```
docker pull nginx:latest           # Pull from Docker Hub
docker tag myapp:latest myusername/myapp:1.0 # Tag for push
docker login                       # Authenticate
docker push myusername/myapp:1.0    # Push to Docker Hub
```

Alternatives:

- AWS ECR, Google Artifact Registry, Azure ACR — cloud-native registries
- GitHub Container Registry (ghcr.io) — integrated with GitHub Actions
- Harbor — open-source, self-hosted
- JFrog Artifactory — enterprise on-prem

6. How do you list, stop, and remove containers?

Answer:

```
docker ps           # List running containers
docker ps -a        # List all (including stopped)
docker stop <container_id> # Graceful stop (SIGTERM → SIGKILL after 10s)
docker rm <container_id>  # Remove stopped container
docker rm -f <container_id> # Force remove running container
docker container prune  # Remove all stopped containers
```

7. What is the difference between `docker stop` and `docker kill`?

Answer:

Command	Signal	Behavior
docker stop	SIGTERM → SIGKILL (after 10s)	Graceful — app can clean up, close DB connections
docker kill	SIGKILL immediately	Forceful — process killed instantly, no cleanup

Always prefer `docker stop` in production so in-flight requests complete and connections close properly.

8. What is a Docker Volume? Why is it needed?

Answer:

Container filesystems are **ephemeral** — all data is lost when a container is removed. Volumes provide persistent storage outside the container lifecycle.

Type	Command	Use Case
Named Volume	docker run -v mydata:/app/data	Production DBs, managed by Docker
Bind Mount	docker run -v /host/path:/container/path	Dev mode live-reload

tmpfs	docker run --tmpfs /tmp	Sensitive temp data, not persisted
-------	-------------------------	------------------------------------

```
docker volume create mydata
docker run -v mydata:/var/lib/postgresql/data postgres
/var/lib/docker/volumes/[volume_name]/_data
```

9. What is the difference between `COPY` and `ADD` in a Dockerfile?

Answer:

Feature	COPY	ADD
Copy local files	Yes	Yes
Fetch from URL	No	Yes
Auto-extract .tar.gz	No	Yes
Recommended	For most cases	Only when extraction needed

Best practice: Use `COPY` unless you specifically need `ADD`'s extra features. For URLs, use `RUN curl` instead — it gives better layer caching control.

10. What does `docker exec` do? How is it different from `docker run` and `docker attach`?

Answer:

Command	Behavior
docker run	Creates and starts a new container from an image
docker exec -it <id> bash	Runs a new process inside an already running container. Exiting does NOT stop the container.
docker attach <id>	Connects to the main PID 1 process. Pressing Ctrl+C stops the container.

```
docker run -d nginx
docker exec -it mynginx /bin/bash # Debug - safe, won't stop container
```

Always use `docker exec` for debugging — never `docker attach` in production.

11. What is a Docker Registry? What are the components of a Docker image name?

Answer:

A registry is a storage and distribution system for Docker images.

- Public: Docker Hub, GitHub Container Registry (ghcr.io)
- Private/Cloud: AWS ECR, Azure ACR, Google GCR
- Self-hosted: Harbor, JFrog Artifactory

Image name anatomy:

```
123456.dkr.ecr.us-east-1.amazonaws.com / myapp : v1.0
└──────────┬──────────┬──────────┬──────────┬──────────┘
registry      repo      tag
```

Default registry is `docker.io` (Docker Hub) if not specified. Default tag is `latest`.

12. How do you view logs of a running container?

Answer:

```
docker logs <container_id>           # All logs
docker logs -f <container_id>        # Follow / live tail
docker logs --tail 100 <container_id> # Last 100 lines
docker logs --since 10m <container_id> # Logs from last 10 minutes
```

Section 2: Docker — Intermediate

13. What is a Multi-Stage Docker Build and why is it important?

Answer:

Multi-stage builds use multiple `FROM` statements in one Dockerfile. Each stage can copy artifacts from the previous stage, leaving behind build tools and intermediate files.

Why it matters:

- Drastically reduces image size — only runtime in final image, not build tools
- Improves security — no Maven, JDK, npm, or source code in production image
- Single Dockerfile — no separate build scripts

Example from this project's backend:

```
# Stage 1: Build (Maven + JDK + source → ~800MB)
FROM maven:3.9-eclipse-temurin-17 AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B          # Cache dependencies in separate
layer
COPY src/ ./src/
RUN mvn package -DskipTests -B

# Stage 2: Runtime (JRE only → ~250MB)
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
USER appuser
ENTRYPOINT ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

Result: **~800MB** → **~250MB** — 68% size reduction.

14. How do you reduce Docker image size?

Answer:

1. **Use minimal base images** — `alpine`, `distroless`, `slim` or `scratch` instead of `ubuntu`
2. **Multi-stage builds** — exclude build tools from the final image
3. **Combine `RUN` commands** — each instruction creates a layer; cleanup must be in the same layer

```
# Bad — cleanup layer doesn't reduce size
RUN apt-get update
RUN apt-get install -y curl
```

```

RUN rm -rf /var/lib/apt/lists/*

# Good - single layer, cleanup works
RUN apt-get update && apt-get install -y curl && rm -rf
/var/lib/apt/lists/*

```

4. Use `.dockerignore` — exclude `node_modules/`, `.git/`, `tests`, `docs`

5. JRE not JDK for Java — saves ~400MB

6. Remove dev dependencies — `npm install --omit=dev`

15. How does Docker layer caching work? How do you optimize it?

Answer:

Each instruction creates a read-only **layer**. Layers are cached and reused if unchanged. If layer N changes, all subsequent layers are invalidated.

Rule: put things that change rarely near the top.

```

# Bad - any code change invalidates npm install cache
COPY . .
RUN npm install

# Good - package.json rarely changes → npm install layer almost always
cached
COPY package*.json ./
RUN npm install
COPY src/ ./src/

```

From this project's backend:

```

COPY pom.xml .
RUN mvn dependency:go-offline -B # Cached until pom.xml changes
COPY src/ ./src/
RUN mvn package -DskipTests -B

```

16. What is `.dockerignore` and why is it important?

Answer:

`.dockerignore` excludes files from the build context sent to the Docker daemon — similar to `.gitignore`.

Why it matters: Faster builds, smaller images, prevents accidentally including `.env` or credentials.

```

node_modules/
target/
*.class
*.jar
.git/
.env
*.log
.idea/

```

17. What happens when you run `docker build`?

Answer:

1. Docker reads the Dockerfile top to bottom

2. For each instruction, checks the **layer cache** — if cache hit, reuses it
 3. On **cache miss**, executes the instruction and creates a new image layer
 4. Produces a final image with all layers stacked (union filesystem)
 5. Image stored locally, can be tagged and pushed to a registry
- Build context is first sent to the Docker daemon as a tar archive — this is why `.dockerignore` matters for large repos.

18. How do you pass environment variables to a Docker container?

Answer:

```
# 1. At runtime
docker run -e DB_HOST=localhost -e DB_PASSWORD=secret myimage

# 2. From a file
docker run --env-file .env myimage

# 3. In Dockerfile (baked in — NOT for secrets)
ENV DB_HOST=localhost
```

Security rule: Never bake secrets into images. Use runtime injection or secret management (Vault, AWS Secrets Manager, K8s Secrets).

19. What is Docker Compose and how does it differ from Kubernetes?

Answer:

Aspect	Docker Compose	Kubernetes
Purpose	Multi-container apps on single host	Orchestration across a cluster
Scaling	Basic (<code>--scale</code>)	Advanced (HPA, Cluster Autoscaler)
Self-healing	No	Yes (restarts failed pods)
Use case	Local development	Production at scale

20. Explain Docker networking modes.

Answer:

Mode	Description	Use Case
bridge (default) - clusterip	Private internal network	Single-host multi-container apps
Host - nodeport	Container shares host network stack	High performance, no NAT overhead
none	No networking	Isolated batch jobs
overlay	Spans multiple Docker hosts	Docker Swarm, multi-host
macvlan	Container gets its own MAC/IP on LAN	Legacy apps needing direct LAN access

```
docker network create banking-net
docker run --network banking-net --name backend ...
docker run --network banking-net --name frontend ...
# frontend can reach backend at http://backend:8080
```

Section 3: Docker — Advanced

22. What are Docker namespaces and cgroups?

Answer:

These are the Linux kernel features that make containers possible.

Namespaces provide **isolation** — each container sees its own:

pid (processes), net (network), mnt (filesystem), uts (hostname), ipc, user

cgroups (Control Groups) provide **resource limiting**:

```
docker run --memory=512m --cpus=0.5 myimage
# This creates cgroup rules - JVM/app cannot exceed these
```

23. How do you scan a Docker image for vulnerabilities?

Answer:

```
docker scout cves myimage:v1      # Docker Scout (built-in)
trivy image myimage:v1            # Trivy (most popular in CI)
 snyk container test myimage:v1   # Snyk
```

In GitHub Actions:

```
- uses: aquasecurity/trivy-action@master
with:
  image-ref: 'myrepo/banking-backend:v1'
  severity: 'CRITICAL,HIGH'
  exit-code: '1'    # Fail pipeline if critical vulns found
```

Section 4: Kubernetes — Basics

27. What is Kubernetes and what problems does it solve?

Answer:

Kubernetes (K8s) is an open-source **container orchestration platform** originally built by Google, now maintained by CNCF.

Problems it solves:

- Self-healing — restarts failed containers, reschedules pods on failed nodes
- Auto-scaling — scales pods up/down based on CPU/memory (HPA)
- Load balancing — distributes traffic across pod replicas
- Rolling updates — zero-downtime deployments and instant rollbacks
- Service discovery — pods find each other by DNS name, not fragile IPs
- Config management — ConfigMaps/Secrets decouple config from images
- Storage orchestration — automatically mounts cloud or local storage

28. Explain the Kubernetes architecture — control plane and worker nodes.

Answer:

Control Plane:

Component	Role
kube-apiserver	Front door — all kubectl commands go here
etcd	Distributed key-value store — single source of truth for all cluster state
kube-scheduler	Assigns unscheduled pods to nodes (filtering + scoring)
kube-controller-manager	Runs Deployment, ReplicaSet, Node, Job controllers
cloud-controller-manager	Interfaces with cloud provider APIs (AWS, GCP, Azure)

Worker Nodes:

Component	Role
kubelet	Agent on each node — ensures containers run as specified
kube-proxy	Manages network rules (iptables/ipvs) for Service routing
Container Runtime	Runs containers (containerd, CRI-O)

29. What is a Pod? Can a Pod have multiple containers?

Answer:

A Pod is the **smallest deployable unit** in Kubernetes — wraps one or more containers sharing the same network namespace (same IP, communicate via localhost), storage volumes, and lifecycle.

Multi-container Pod patterns:

- Sidecar: helper alongside main app — e.g., log shipper (Fluentd), Envoy proxy
- Init container: runs to completion before main containers start — e.g., wait for DB
- Ambassador: proxy for external communication

```
spec:
  initContainers:
  - name: wait-for-db
    image: busybox
    command: ['sh', '-c', 'until nc -z postgres 5432; do sleep 2; done']
  containers:
  - name: backend
    image: banking-backend:v1
```

30. What is the difference between a Deployment, ReplicaSet, StatefulSet, and DaemonSet?

Answer:

Object	Pod Identity	Use Case
Deployment	Random names, interchangeable	Stateless apps: web APIs, microservices
ReplicaSet	Random names (managed by Deployment)	Rarely used directly

StatefulSet	Stable identity (db-0, db-1), own PVC each	Databases (PostgreSQL, MongoDB, Kafka)
DaemonSet	One pod per node, auto-added to new nodes	Node agents: Fluentd, node-exporter, Falco

DaemonSet: Ensures one pod runs on EVERY node. When a new node joins the cluster, it automatically gets the DaemonSet pod.

31. What is the role of a ReplicaSet? How does it relate to a Deployment?

Answer:

- ReplicaSet ensures a specified number of identical Pod replicas are always running
- Deployment manages ReplicaSets — on update, creates a new ReplicaSet and scales it up while scaling the old one down. Old ReplicaSet is kept for rollback.

```
Deployment
├── ReplicaSet v2 (current) → Pod, Pod, Pod
└── ReplicaSet v1 (old, 0 pods – kept for rollback)
```

Never create a ReplicaSet directly — always use a Deployment.

32. What are the Kubernetes Service types? Explain each with use cases.

Answer:

Type	Accessibility	Use Case
ClusterIP (default)	Within cluster only	Internal service-to-service
NodePort	Each node's IP at static port (30000-32767)	Dev/testing, Minikube
LoadBalancer	Cloud load balancer (AWS ALB/NLB)	Production internet-facing
ExternalName	CNAME to external DNS	Pointing to external databases

In this project:

- PostgreSQL → ClusterIP (only backend should reach it)
- Backend → ClusterIP (only frontend should reach it)
- Frontend → NodePort :30080 (user browser access in Minikube)

33. What is a Namespace? Why would you use multiple namespaces?

Answer:

Namespaces provide **virtual clusters** within a physical cluster.

Use cases:

- Environment isolation: dev, staging, production in one cluster
- Team isolation: team-payments, team-auth with separate RBAC
- Resource quotas: limit CPU/memory per namespace

This project uses `banking-app` namespace to isolate all resources.

```
kubectl get pods -n banking-app
kubectl get pods --all-namespaces
kubectl config set-context --current --namespace=banking-app
```

34. What is a ConfigMap and a Secret? How do you consume them in a Pod?

Answer:

Aspect	ConfigMap	Secret
Data	Plain text	Base64-encoded
Use case	DB hostname, port, feature flags	Passwords, API keys, TLS certs
Encryption at rest	No	Optional (with etcd KMS provider)

Three ways to consume:

```
# 1. Individual env vars
env:
  - name: DB_HOST
    valueFrom:
      configMapKeyRef: { name: backend-config, key: DB_HOST }

# 2. All keys at once
envFrom:
  - configMapRef: { name: backend-config }
  - secretRef:    { name: postgres-secret }

# 3. Volume mount (auto-updates without pod restart)
volumeMounts:
  - mountPath: /etc/config
    name: config-volume
```

Important: Base64 is encoding, not encryption. Use Sealed Secrets or Vault for real security.

35. What is a PersistentVolume (PV) and PersistentVolumeClaim (PVC)?

Answer:

Concept	Analogy	Description
PV	A physical hard drive	Actual storage (NFS, EBS, GCE disk) provisioned by admin
PVC	A request form for storage	Pod's request for a certain size and access mode
StorageClass	A storage catalog	Enables dynamic provisioning

PVC created → K8s binds to matching PV (or dynamically provisions) → Pod mounts PVC

In this project, PostgreSQL uses a PVC so data survives pod restarts and rescheduling.

36. Name 5 essential `kubectl` commands used daily.

Answer:

```
kubectl get pods -n <namespace>          # List pods and status
kubectl describe pod <name> -n <namespace> # Detailed info + Events
kubectl logs <pod> --tail=100 -n <namespace> # View container logs
kubectl apply -f manifest.yaml             # Create/update resources
kubectl exec -it <pod> -- /bin/sh          # Shell into running pod
kubectl rollout status deployment/<name>    # Check rollout progress
```

```
kubectl rollout undo deployment/<name>      # Rollback to previous version
kubectl top pods -n <namespace>              # CPU/memory usage
kubectl port-forward svc/<name> 8080:8080     # Local access to service
kubectl get events -n <namespace> --sort-by='.lastTimestamp'
```

37. What is the role of etcd in Kubernetes?

Answer:

etcd is a distributed key-value store and the **single source of truth** for all cluster state — every Pod, Service, Deployment, Secret, ConfigMap is stored here.

- All API server writes are persisted to etcd
- All controllers watch etcd for changes
- etcd failure = cluster cannot make scheduling decisions — but existing pods keep running (kubelet is autonomous)
- Production: run as 3- or 5-node cluster (odd number for quorum)
- Always snapshot regularly: `etcdctl snapshot save backup.db`

38. What happens when you run `kubectl apply -f deployment.yaml`?

Answer:

1. `kubectl` sends HTTP request to `kube-apiserver`
2. API server **authenticates** (certificate/token) and **authorizes** (RBAC check)
3. Validates YAML schema, runs **admission controllers** (LimitRanger, OPA Gatekeeper, etc.)
4. Persists desired state to **etcd**
5. **Deployment controller** detects change, creates/updates a ReplicaSet
6. **ReplicaSet controller** creates Pod objects in etcd
7. **Scheduler** assigns each pending pod to a node (filtering → scoring)
8. **kubelet** on assigned node pulls the image, starts the container
9. kubelet reports pod status back to API server

Section 5: Kubernetes — Intermediate

39. What are liveness, readiness, and startup probes? How do they differ?

Answer:

Probe	Question Asked	Action on Failure
Readiness	"Is this pod ready to receive traffic?"	Removed from Service endpoints — no traffic sent
Liveness	"Is this pod still alive/not deadlocked?"	Container restarted
Startup	"Has the app started yet?"	Container killed if not started by deadline

Startup probe disables liveness/readiness until it succeeds — prevents premature restarts for slow-starting apps.

From this project's backend (Spring Boot):

```
readinessProbe:
  httpGet: { path: /actuator/health, port: 8080 }
  initialDelaySeconds: 30    # Spring Boot needs 20-40s to start
  periodSeconds: 10

livenessProbe:
  httpGet: { path: /actuator/health, port: 8080 }
  initialDelaySeconds: 45    # Higher than readiness - prevents restart loop
  periodSeconds: 15
```

40. How does Kubernetes handle rolling updates? What are `maxSurge` and `maxUnavailable`?

Answer:

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1          # Max EXTRA pods above desired during update
      maxUnavailable: 0    # Max pods that can be unavailable during update
```

With `maxSurge: 1`, `maxUnavailable: 0`, `replicas: 3`:

1. Create 1 new pod v2 (now 4 pods — surge allowed)
2. New pod passes readiness probe → terminate 1 old pod (back to 3)
3. Repeat until all 3 are v2 → **zero downtime**

```
kubectl set image deployment/backend backend=myrepo/banking-backend:v2 -n banking-app
kubectl rollout status deployment/backend -n banking-app
kubectl rollout undo deployment/backend -n banking-app    # Rollback
kubectl rollout history deployment/backend -n banking-app
```

41. What is RBAC in Kubernetes? How do you configure it?

Answer:

Object	Scope	Description
Role	Namespace	Permissions within a namespace
ClusterRole	Cluster-wide	Permissions across all namespaces
RoleBinding	Namespace	Assigns Role to user/group/ServiceAccount
ClusterRoleBinding	Cluster-wide	Assigns ClusterRole cluster-wide

```
kind: Role
metadata: { name: pod-reader, namespace: production }
rules:
- apiGroups: [""]
  resources: ["pods", "pods/log"]
  verbs: ["get", "list", "watch"]
---
kind: RoleBinding
subjects:
```

```
- { kind: User, name: jane@company.com }
  roleRef: { kind: Role, name: pod-reader }
```

Best practice: Principle of least privilege. CI/CD pipelines use ServiceAccounts scoped to their namespace.

42. What is an Ingress? How does it differ from a LoadBalancer Service?

Answer:

	LoadBalancer Service	Ingress
Load balancers	One per service — expensive	Single entry point for all services
Routing	Port-based only	Host and path-based
TLS	Per service	Centralized termination

```
spec:
  rules:
  - host: banking.example.com
    http:
      paths:
      - path: /api
        backend: { service: { name: backend, port: { number: 8080 } }}
      - path: /
        backend: { service: { name: frontend, port: { number: 3000 } }}
```

Requires an **Ingress Controller**: nginx-ingress, AWS ALB Ingress Controller, Traefik.

Gateway API is the modern replacement with better role separation between infra and app teams.

43. What is a HorizontalPodAutoscaler (HPA)?

Answer:

HPA automatically scales pod replicas based on observed metrics.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef: { kind: Deployment, name: backend }
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target: { type: Utilization, averageUtilization: 70 }

kubectl autoscale deployment backend --cpu-percent=70 --min=2 --max=10
```

Prerequisite: metrics-server must be installed. For custom metrics (HTTP req/s, queue depth), use **Prometheus Adapter**.

44. What are resource requests and limits? What are QoS classes?

Answer:

Field	Meaning	Effect if Exceeded
requests.cpu	Minimum CPU guaranteed	Used by scheduler

requests.memory	Minimum memory guaranteed	Used by scheduler
limits.cpu	Maximum CPU	Container throttled
limits.memory	Maximum memory	Container OOMKilled

QoS Classes:

- **Guaranteed:** requests == limits — never evicted under pressure
- **Burstable:** requests < limits — evicted second
- **BestEffort:** no requests/limits — evicted first

```
resources:
  requests: { cpu: 200m, memory: 256Mi }
  limits:   { cpu: 500m, memory: 512Mi }
```

45. What happens when a Kubernetes node goes down?

Answer:

1. Node stops sending heartbeats to API server
2. After `node-monitor-grace-period` (**40s**) → node marked `NotReady`
3. After `pod-eviction-timeout` (**5 minutes**) → pods evicted from the node
4. Pods managed by Deployments/ReplicaSets rescheduled on healthy nodes
5. **Bare pods (no controller) are NOT rescheduled** — always use Deployments

46. How does Kubernetes DNS work internally?

Answer:

CoreDNS runs in `kube-system`. Every pod has `/etc/resolv.conf` pointing to CoreDNS ClusterIP.

DNS format: `<service-name>.<namespace>.svc.cluster.local`

```
curl http://backend:8080 # Within same namespace
curl http://backend.banking-app.svc.cluster.local:8080 # Cross-namespace
```

In this project: Frontend uses `BACKEND_URL=http://backend:8080`. CoreDNS resolves `backend` → ClusterIP → kube-proxy routes to a healthy pod.

Troubleshooting:

```
kubectl run dns-test --image=busybox -it --rm -- nslookup backend
kubectl get endpoints backend -n banking-app # Empty = selector labels don't match pods!
```

47. How do resource requests affect pod scheduling?

Answer:

The scheduler checks each node:

```
node.allocatable - sum(existing pod requests) >= new_pod.requests
```

If no node satisfies this, the pod stays **Pending**.

- Limits do NOT affect scheduling — only requests do
- Over-requesting wastes node capacity; under-requesting risks OOM

```
kubectl describe node minikube | grep -A5 "Allocated resources"
kubectl top nodes
```

Section 6: Kubernetes — Advanced

48. How does the Kubernetes scheduler decide where to place a Pod?

Answer:

Two phases:

1. **Filtering:** Eliminates nodes that don't satisfy requirements:
 - Sufficient CPU/memory (requests), node selector/affinity, taints tolerated, PV topology
2. **Scoring:** Ranks remaining nodes:
 - Least requested (spread load), image locality, inter-pod affinity/anti-affinity

Pod is bound to the highest-scoring node. You can write custom schedulers or extend via scheduler plugins.

49. Explain taints, tolerations, node affinity, and pod anti-affinity.

Answer:

Taints — repel pods from a node:

```
kubectl taint nodes gpu-node gpu=true:NoSchedule
# Effects: NoSchedule, PreferNoSchedule, NoExecute (evicts existing pods
too)
```

Tolerations — allow a pod onto a tainted node:

```
tolerations:
- { key: "gpu", operator: "Equal", value: "true", effect: "NoSchedule" }
```

Node Affinity — Tell pods *where to go*

```
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
      - matchExpressions:
        - { key: zone, operator: In, values: ["us-east-1a"] }
```

Pod Anti-Affinity — Tell pods *where NOT to go (relative to other pods)*

```
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      - labelSelector:
          matchLabels: { app: backend }
        topologyKey: kubernetes.io/hostname # No two backend pods on same
node
```

50. What is the Kubernetes API server authentication and authorization flow?

Answer:

Every request goes through three gates:

1. **Authentication (Who are you?):** Client certificates, bearer tokens (ServiceAccount JWT, OIDC), webhook
2. **Authorization (What can you do?):** RBAC (most common), ABAC, Webhook, Node authorizer
3. **Admission Control (Should this be allowed?):**
 - Mutating webhooks — modify the request (inject sidecars, set defaults)
 - Validating webhooks — accept or reject (OPA Gatekeeper policies)
 - Built-in: LimitRanger, ResourceQuota, PodSecurity

51. What is a Kubernetes Admission Controller? Give a real example.

Answer:

Admission controllers intercept API requests **after auth/authz but before persisting to etcd**. They can mutate or validate requests.

Built-in: LimitRanger (default resource limits), ResourceQuota (namespace quotas), PodSecurity (enforce non-root, no privilege escalation).

Custom (OPA Gatekeeper):

```
# Policy: reject pods missing 'app' and 'team' labels
rego: |
violation[{"msg": msg}] {
  required := {"app", "team"}
  missing := required - {k | input.review.object.metadata.labels[k]}
  count(missing) > 0
  msg := sprintf("Missing required labels: %v", [missing])
}
```

52. What is the difference between Recreate and RollingUpdate deployment strategies?

Answer:

Strategy	Behavior	Downtime	Use Case
RollingUpdate (default)	Gradually replaces pods	Zero	Standard updates
Recreate	Kills ALL old pods, then creates new	Yes (brief)	Old and new can't coexist (DB schema migration)

Advanced via Argo Rollouts:

- Canary: Route 10% traffic to new version → validate → increase gradually
- Blue/Green: Run two full environments, switch traffic instantly

53. What is a Pod Disruption Budget (PDB)?

Answer:

A PDB limits pods that can be down simultaneously during **voluntary disruptions** (node drains, cluster upgrades).

```
apiVersion: policy/v1
kind: PodDisruptionBudget
```

```
spec:
  minAvailable: 2      # Always keep at least 2 running
  selector:
    matchLabels: { app: backend }
```

Real scenario: Without PDB, a cluster upgrade drained two nodes simultaneously and evicted all 3 backend pods, causing outage. With `minAvailable: 2`, the second drain was blocked until pods from the first node were healthy again.

Watch out: `minAvailable == total replicas` blocks ALL evictions — nodes can never be drained.

54. What is a service mesh? What problems does it solve?

Answer:

A service mesh (Istio, Linkerd, Cilium) adds an infrastructure layer via sidecar proxies (Envoy) for service-to-service communication.

Problems it solves that Kubernetes alone doesn't:

- mTLS — automatic encryption between services, zero-trust networking
- Traffic management — canary releases, circuit breaking, retries, timeouts
- Distributed tracing — request traces across microservices without code changes
- Observability — automatic metrics per service (latency, error rate, throughput)

55. How do you implement Network Policies in Kubernetes?

Answer:

```
# Only allow backend to receive traffic from frontend, send to postgres
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: { name: backend-policy, namespace: banking-app }
spec:
  podSelector:
    matchLabels: { app: backend }
  policyTypes: [Ingress, Egress]
  ingress:
  - from:
    - podSelector: { matchLabels: { app: frontend } }
    ports: [{ port: 8080 }]
  egress:
  - to:
    - podSelector: { matchLabels: { app: postgres } }
    ports: [{ port: 5432 }]
  - to: []          # Always allow DNS
    ports: [{ port: 53, protocol: UDP }]
```

Critical: NetworkPolicies only work if your CNI supports them — **Calico, Cilium** do. Default AWS VPC CNI does NOT. Default behavior without any policy: **all traffic is allowed**.

56. How do you drain a node safely before maintenance?

Answer:

```
kubectl cordon <node-name>      # Prevent new pods from scheduling here
```

```

kubect1 drain <node-name> \
--ignore-daemonsets \           # Can't evict DaemonSet pods - skip them
--delete-emptydir-data \       # Allow evicting pods with emptyDir
--grace-period=60 \            # Give pods 60s to finish in-flight
requests
--timeout=300s

# Do maintenance...

kubect1 uncordon <node-name>    # Allow pods to schedule here again

```

If a pod is stuck in Terminating: check if a PDB is blocking eviction with `kubect1 get pdb`. Last resort: `kubect1 delete pod <pod> --force --grace-period=0`.

57. How would you design a multi-tenant Kubernetes cluster?

Answer:

Soft multi-tenancy (same cluster):

- Separate namespaces per team/environment
- NetworkPolicies blocking cross-namespace traffic by default
- ResourceQuotas and LimitRanges per namespace
- RBAC — teams access only their namespaces
- OPA/Gatekeeper admission policies

Hard multi-tenancy (untrusted tenants):

- vcluster — each tenant gets a full virtual K8s API server on shared nodes
- Separate node pools with taints/tolerations per tenant
- gVisor or Kata Containers for kernel-level isolation

Section 7: Scenario-Based / Troubleshooting

58. Your deployment rolled out but half the pods return 500 errors. What do you do?

Answer:

```

# 1. Confirm it was a recent rollout
kubect1 rollout history deployment/backend -n banking-app

# 2. Check if old and new pods are running simultaneously
kubect1 get pods -n banking-app

# 3. Check new pod logs for exceptions
kubect1 logs <new-pod> -n banking-app --tail=100

# 4. Check readiness probe failures
kubect1 describe pod <new-pod> -n banking-app

# 5. Immediate rollback
kubect1 rollout undo deployment/backend -n banking-app

```

```
# 6. Check if new version needs a new ConfigMap/Secret not yet applied
kubectl get configmap,secret -n banking-app
```

59. A Pod is stuck in `Pending` state for 10+ minutes. How do you diagnose?

Answer:

```
kubectl describe pod <pod-name> -n banking-app
# Events section almost always tells you the exact reason
```

Reason	Event Message	Fix
Insufficient CPU/memory	0/5 nodes available: Insufficient cpu	Scale up nodes or reduce requests
Node selector mismatch	didn't match Pod's node affinity	Fix node labels or relax affinity
Taint not tolerated	had untolerated taint	Add toleration or remove taint
PVC not bound	Pod waits for storage	kubectl get pvc — check StorageClass
ResourceQuota exceeded	exceeded quota	kubectl describe quota -n banking-app

60. A Pod is in `CrashLoopBackOff`. Give your step-by-step approach.

Answer:

```
# Step 1: Events and last state
kubectl describe pod <pod-name> -n banking-app
# Check: Events, Last State, Reason, Exit Code

# Step 2: Current logs
kubectl logs <pod-name> -n banking-app

# Step 3: PREVIOUS container logs — logs from before the crash
kubectl logs <pod-name> -n banking-app --previous

# Step 4: Interpret exit code
# Exit 1 = Application error (check logs for exception)
# Exit 137 = OOMKilled (increase memory limits)
# Exit 143 = SIGTERM (liveness probe killing it — check probe config)
# Exit 0 = Container completed (wrong for a long-running service)

# Step 5: Debug — override command to keep container alive
kubectl debug <pod-name> -it --image=busybox -- sh
```

Common root causes: wrong DB connection env var, missing Secret/ConfigMap, OOMKilled, misconfigured liveness probe, port mismatch, volume permission issues.

61. A Pod is in `Error` state. Walk through the commands you run.

Answer:

```
kubectl get pods -n banking-app # Note STATUS and RESTARTS

kubectl describe pod <pod-name> -n banking-app # Events, Init Containers, Exit Code

kubectl logs <pod-name> --all-containers=true -n banking-app
kubectl logs <pod-name> --previous -n banking-app
```

```
# If init containers exist, check those too
kubectl logs <pod-name> -c init-container-name -n banking-app

# Test connectivity from a debug pod
kubectl run debug --image=curlimages/curl -it --rm -- sh
curl http://postgres:5432
```

62. Your Spring Boot backend pod keeps getting 'OOMKilled'. How do you diagnose and fix it?

Answer:

```
kubectl describe pod <pod-name> -n banking-app | grep -A3 "Last State"
# Shows: Reason: OOMKilled

kubectl top pods -n banking-app # Check current memory usage
```

Immediate fix:

```
resources:
limits: { memory: "1Gi" } # Increase from 512Mi
```

Java-specific root cause fix (in Dockerfile):

```
ENV JAVA_OPTS="-XX:+UseContainerSupport -XX:MaxRAMPercentage=75.0"
```

- UseContainerSupport — JVM respects cgroup limit, not total host RAM
- MaxRAMPercentage=75 — heap is max 75% of container limit (leaves room for off-heap, metaspace, GC overhead)

63. How would you do an EKS cluster upgrade from version 1.27 to 1.28?

Answer:

Never skip versions — upgrade one minor version at a time.

Pre-upgrade:

- Check 1.28 changelog for deprecated/removed APIs
- Run pluto detect-all-in-cluster to find deprecated API usage
- Test in non-prod first

```
# Step 1: Control plane (takes 15-25 min)
aws eks update-cluster-version --name my-cluster --kubernetes-version 1.28

# Step 2: Update add-ons
aws eks update-addon --cluster-name my-cluster --addon-name vpc-cni --
addon-version v1.16.0-eksbuild.1
aws eks update-addon --cluster-name my-cluster --addon-name coredns --
addon-version v1.10.1-eksbuild.6

# Step 3: Upgrade node groups (rolling — drains and replaces nodes)
aws eks update-nodegroup-version --cluster-name my-cluster --nodegroup-name
standard-workers
```

64. How do you run a scheduled batch job in Kubernetes?

Answer:

```
apiVersion: batch/v1
kind: CronJob
```

```

  metadata: { name: daily-report, namespace: banking-app }
  spec:
    schedule: "0 0 * * *"           # Daily at midnight
    concurrencyPolicy: Forbid        # Don't run if previous job still running
    successfulJobsHistoryLimit: 3
    failedJobsHistoryLimit: 1
    jobTemplate:
      spec:
        template:
          spec:
            restartPolicy: OnFailure
            containers:
            - name: reporter
              image: myrepo/reporter:v1

  kubectl create job --from=cronjob/daily-report manual-run # Trigger manually

```

65. How do you implement HPA for a custom metric (e.g., requests per second)?

Answer:

Install **Prometheus Adapter** to expose Prometheus metrics to the Kubernetes Custom Metrics API, then reference them in HPA:

```

apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef: { kind: Deployment, name: backend }
  minReplicas: 2
  maxReplicas: 20
  metrics:
  - type: Pods
    pods:
      metric: { name: http_requests_per_second }
      target: { type: AverageValue, averageValue: 1000m } # 1 req/s per
pod

```

66. How do you manage secret rotation in Kubernetes?

Answer:

Use **External Secrets Operator (ESO) + AWS Secrets Manager**:

```

apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
spec:
  refreshInterval: 1h           # Sync every hour from AWS Secrets Manager
  secretStoreRef: { name: aws-secrets-manager }
  target: { name: postgres-secret }
  data:
  - secretKey: POSTGRES_PASSWORD
    remoteRef: { key: banking-app/db-password }

```

Pods picking up the rotated secret:

- Env vars: Pod must restart. Use Stakater Reloader — watches Secrets and triggers rolling restart automatically
- Volume-mounted secrets: K8s updates the file within ~1 minute — apps that re-read the file work without restart (ideal for TLS certs)

67. How would you expose this banking application to the internet securely?

Answer:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  tls:
  - hosts: ["banking.example.com"]
    secretName: banking-tls
  rules:
  - host: banking.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service: { name: frontend, port: { number: 3000 } }
```

Security layers: TLS at Ingress, NetworkPolicies to restrict pod-to-pod traffic, no direct DB access from outside cluster, Secrets managed via Vault/ESO.

68. How would you configure this app for multiple environments (dev/staging/prod)?

Answer:

Use environment-specific ConfigMaps per namespace — no code changes needed:

```
# Dev
kubectl create configmap frontend-config \
  --from-literal=BACKEND_URL=http://backend:8080 -n dev

# Prod
kubectl create configmap frontend-config \
  --from-literal=BACKEND_URL=http://backend:8080 -n production
```

Better: Use Helm with values files or Kustomize overlays:

```
k8s/
├── base/                # Common manifests
└── overlays/
    ├── dev/             # Dev-specific patches
    ├── staging/
    └── production/      # Prod-specific patches (more replicas,
higher limits)
```

Section 8: Project-Specific — Banking App on K8s

69. Walk me through the architecture of your three-tier banking application on Kubernetes.

Answer:

"The application is a three-tier architecture running on Minikube:

- Tier 1 — Frontend: Node.js Express + EJS on port 3000. Exposed via NodePort :30080. Renders the banking UI and calls the backend REST API.
- Tier 2 — Backend: Java 17 + Spring Boot 3 REST API on port 8080. ClusterIP only — internal access from frontend. Spring Data JPA for DB access. 2 replicas.
- Tier 3 — Database: PostgreSQL 15 on port 5432. ClusterIP only. Data persisted via PVC.

Traffic flow: Browser → NodePort:30080 → Frontend → ClusterIP:8080 → Backend → ClusterIP:5432 → PostgreSQL

Non-sensitive config (DB_HOST, BACKEND_URL) in ConfigMaps. Credentials in Secrets. Spring Boot consumes them via `${ENV_VAR:default}` syntax."

70. Why did you use a multi-stage Docker build for the Java backend? What was the size reduction?

Answer:

"Single-stage with `maven:3.9-eclipse-temurin-17` would produce ~800MB+ including Maven, JDK, dependencies, and source code.

Multi-stage: Stage 1 builds the JAR, Stage 2 uses `eclipse-temurin:17-jre` and copies only the ~50MB JAR.

Result: ~800MB → ~250MB (68% reduction).

Additional optimizations: `mvn dependency:go-offline` caches dependencies separately (code changes don't re-download deps), `-DskipTests` (tests run in CI), `-XX:+UseContainerSupport` (JVM respects cgroup limits), non-root `appuser`."

71. Why does the backend readiness probe use `/actuator/health` instead of `/health`?

Answer:

"Spring Boot Actuator's `/actuator/health` checks the **application AND the database**:

```
{ "status": "UP", "components": { "db": { "status": "UP" }, "diskSpace": { "status": "UP" } }}
```

If PostgreSQL is down → Actuator returns `DOWN` → readiness fails → pod removed from Service endpoints → no 500 errors sent to users.

The custom `/health` endpoint only checks if the JVM is responding, not the database. Using it for readiness would route traffic to pods that can't process requests.

Rule: `/actuator/health` for K8s probes (deep check), `/health` for simple ping."

72. Why does the backend have `initialDelaySeconds: 30` for readiness and `45` for liveness?

Answer:

"Spring Boot startup takes 20-40 seconds: JVM init, Spring context loading, database connection pool, JPA schema migration (`ddl-auto: update`), Actuator registration.

Without delays:

- Readiness fires immediately → fails → pod never becomes Ready
- Liveness fires → fails → pod killed → restart loop → CrashLoopBackOff

Liveness (45s) is set higher than readiness (30s) — if liveness kills before readiness ever passes, you get an infinite restart loop.

Contrast: The Node.js frontend starts in ~1s and uses `initialDelaySeconds: 10.`"

73. Why do Backend and Database use ClusterIP while Frontend uses NodePort?

Answer:

"Security by design — principle of least exposure:

- PostgreSQL (ClusterIP): Database must never be accessible outside the cluster. Only the backend should connect to it.
- Backend API (ClusterIP): REST API is internal. Exposing it externally would allow direct API abuse and bypass frontend validation.
- Frontend (NodePort): Only user-facing entry point. NodePort exposes it for Minikube access. Production would use LoadBalancer or Ingress with TLS.

Security chain enforced by architecture:

```
Internet → Frontend ONLY → Backend (internal) → DB (internal)
```

74. How would you scale this banking application in production?

Answer:

```
# HPA for backend
minReplicas: 2
maxReplicas: 20
metrics:
- type: Resource
resource: { name: cpu, target: { averageUtilization: 70 }}
```

Database scaling:

- Single PostgreSQL pod → single point of failure, can't scale writes
- Production: PostgreSQL with read replicas (StatefulSet or CloudNative PG Operator), or managed DB (AWS RDS Multi-AZ)
- PgBouncer for connection pooling — multiple backend replicas quickly exhaust PostgreSQL's connection limit

Infrastructure: Cluster Autoscaler (add nodes when pods pending), VPA (auto-tune resource requests), PDB (prevent full outage during maintenance).

75. How are secrets stored in this project and how would you improve it?

Answer:

"Currently stored as base64-encoded YAML in Git — anti-pattern because base64 is trivially reversible and etcd is unencrypted by default.

Improvements in order of maturity:

1. **Sealed Secrets:** Encrypt before committing to Git — only cluster can decrypt

```
kubeseal --format yaml < postgres-secret.yaml > sealed-postgres-secret.yaml
```

2. **External Secrets Operator + AWS Secrets Manager:** Store externally, auto-sync to K8s Secrets, rotation handled automatically
 3. **HashiCorp Vault:** Full lifecycle — dynamic secrets, rotation, audit logs, Vault Agent sidecar injection
 4. **Encrypt etcd at rest:** Enable KMS encryption provider in kube-apiserver
- Immediate action:** Add `postgres-secret.yaml` to `.gitignore` — inject via CI/CD pipeline secrets instead."

DevOps Placement Program — Assignment 1 | Interview Q&A Guide

Covers real questions from: Google, Amazon, Netflix, Uber, Flipkart, Razorpay, PayTM, Swiggy, Zomato